

Smart Train

S. Hidalgo Gonzalez¹ y V. Lagos Torres²

^{1,2}Ingeniería Civil Informática. Departamento de ciencias de la computación e informática,
Facultad de Ingeniería, Universidad Andres Bello, Viña del mar.

Asignatura: PTEC102 – Paradigmas de programación - NRC 13046

Docente: Juan Calderon Maureira

Fecha de entrega: 28 de mayo de 2026

RUT: 21.876.861-k y 21.925.820-8



Introducción

La planificación deportiva es una herramienta fundamental en las ciencias del ejercicio para estructurar de manera progresiva y segura los estímulos que un atleta recibe, moviéndose desde niveles de intensidad básicos hasta el alto rendimiento. Sin embargo, como se observa en la gestión diaria de centros deportivos, aplicar esto de forma personalizada es difícil porque en ocasiones no hay un registro claro de la carga previa o cuesta equilibrar el tiempo total de las sesiones con los objetivos físicos del cliente. **Esta tarea nace de la motivación de crear una herramienta práctica para que un deportista pueda armar rutinas de entrenamiento de forma ordenada**, asegurando que cada ejercicio seleccionado cumpla realmente con el nivel de exigencia que se busca trabajar.

El objetivo fundamental de este trabajo es **desarrollar una aplicación de escritorio en Java que funcione como un sistema de gestión de rutinas inteligentes**. El sistema permite gestionar un catálogo de 400 ejercicios cardiovasculares y de fuerza, cada uno con una descripción detallada de su ejecución para garantizar la seguridad del usuario y facilitar el seguimiento de su progreso. Además, el software es capaz de **generar rutinas personalizadas** según el nivel de intensidad que solicite el cliente, evitar la repetición de movimientos en semanas consecutivas y calcular automáticamente el tiempo estimado total que le tomará completar su entrenamiento. Para lograr esto, el desarrollo se enfoca en implementar un código funcional bajo **el paradigma de la programación orientada a eventos** en Java, con una interfaz gráfica desarrollada en Swing que guía al usuario a través del proceso de generación de su rutina de forma intuitiva y visual.

El presente documento se estructura de la siguiente manera, en primer lugar, se describe la solución propuesta detallando las clases diseñadas, las relaciones entre ellas y las decisiones de diseño tomadas, acompañadas del diagrama de clases correspondiente. Finalmente, se presentan las conclusiones del trabajo, con una reflexión sobre los objetivos alcanzados y los aprendizajes obtenidos durante el desarrollo.

Descripción de la solución

La arquitectura del sistema **SmartTrain** se centra en una separación estricta de responsabilidades entre la lógica del negocio y la interfaz gráfica, materializando así uno de los principios fundamentales de la ingeniería de software moderno, el bajo acoplamiento entre capas. Esta decisión arquitectónica se refleja en la organización del proyecto en dos paquetes Java claramente delimitados. El **paquete backend** contiene todas las clases que representan el modelo de datos y las operaciones del sistema, mientras que el **paquete frontend** gestiona exclusivamente las ventanas, componentes visuales y la interacción con el usuario. Gracias a esta separación, cualquier modificación en la interfaz gráfica no afecta la lógica central del sistema, y viceversa, lo que dota al software de una alta cohesión interna y una notable capacidad de escalabilidad a largo plazo. La comunicación entre ambos paquetes se realiza de forma directa pero controlada, el frontend invoca métodos públicos del backend para ejecutar operaciones, y el backend retorna resultados mediante valores de retorno que el frontend interpreta para actualizar la interfaz, manteniendo siempre la separación de responsabilidades entre capas siguiendo las buenas practicas de programacion.

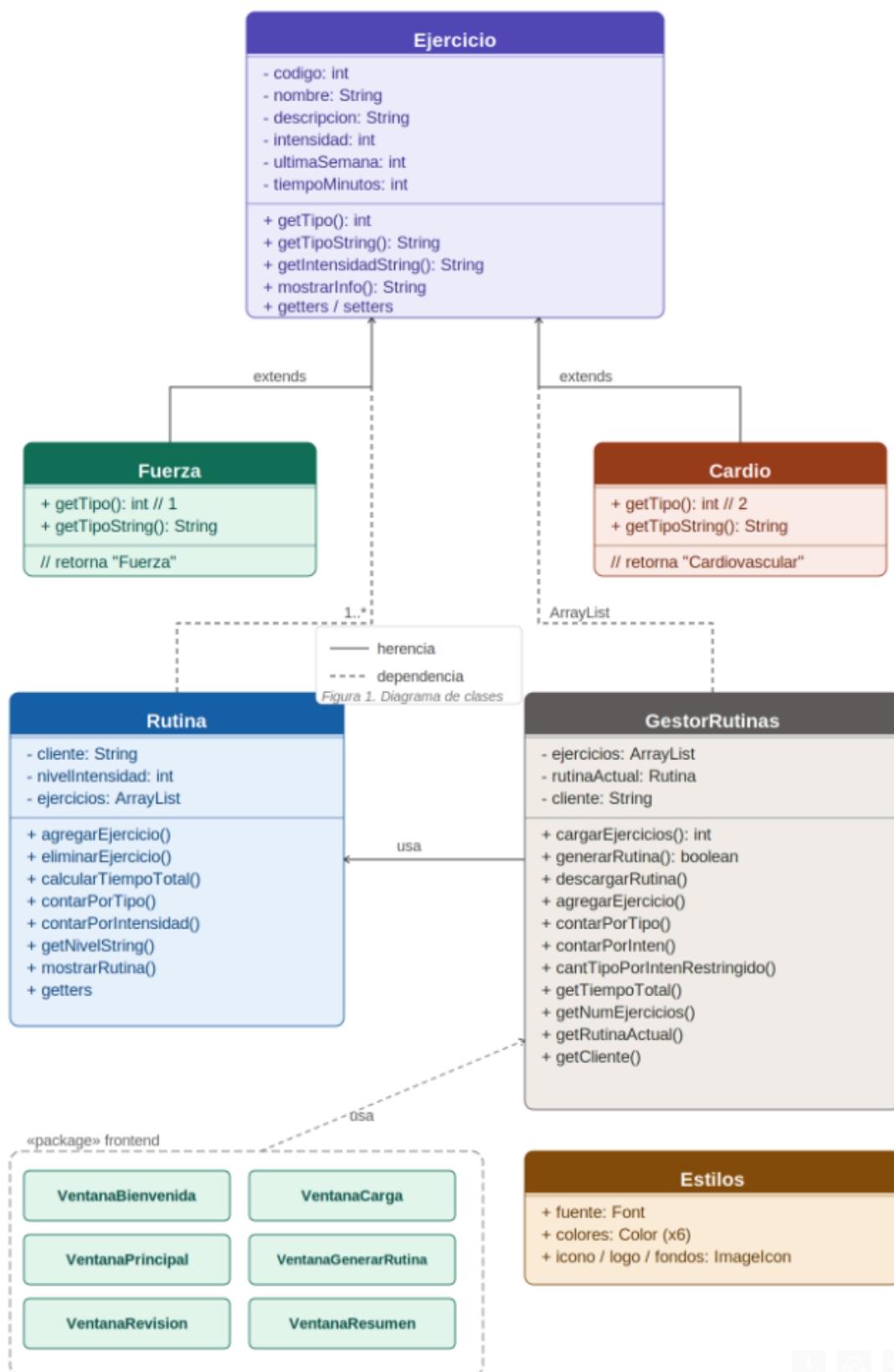
En la cúspide de la jerarquía del backend se define la clase **Ejercicio**, diseñada para encapsular el estado y comportamiento universal de cualquier actividad física del sistema. Esta clase administra seis atributos privados que garantizan el encapsulamiento de la información: el identificador único **código** de tipo entero, el **nombre** como cadena de texto, la **descripción** detallada de ejecución, el **nivel** de intensidad representado como un entero del 1 al 4 donde 1 es Básico, 2 es Intermedio, 3 es Avanzado y 4 es Alto Rendimiento, el atributo **ultimaSemana** que registra si el ejercicio fue o no realizado la semana anterior, donde 1 implica que sí se realizó y 2 indica lo contrario, para aplicar la restricción de no repetición, y por último el **tiempo** estimado en minutos. Todos estos atributos son accesibles únicamente a través de métodos getters y setters públicos, respetando el principio de encapsulamiento. La clase también declara los métodos `getTipo()` y `getTipoString()` como métodos concretos sobrescribibles, equivalentes a los métodos virtuales de C++, permitiendo que las subclases especializadas provean su propia implementación.

A partir de esta clase base derivan dos subclases especializadas. La clase **Fuerza** hereda toda la estructura de **Ejercicio** e implementa `getTipo()` retornando el valor 1 y `getTipoString()` retornando la cadena "Fuerza". La clase **Cardio** sigue el mismo patrón, retornando el valor 2 y la cadena "Cardiovascular" respectivamente. Ambas subclases utilizan la anotación `@Override` para indicar explícitamente al compilador que estos métodos están siendo redefinidos. Esta jerarquía de herencia permite al sistema tratar polimórficamente cualquier ejercicio a través de la referencia de tipo base **Ejercicio**, sin necesidad de conocer su tipo concreto, lo que simplifica la lógica de generación de rutinas y cálculo de estadísticas.

En paralelo a la jerarquía de ejercicios se diseñó la clase **Rutina**, cuya responsabilidad es orquestrar la planificación del entrenamiento para un cliente específico. Esta clase actúa como un contenedor integral que mantiene el **nombre del cliente**, el **nivel de intensidad** seleccionado y la **colección dinámica de ejercicios** asignados implementada mediante `ArrayList<Ejercicio>`, equivalente al vector `<Ejercicio*>` utilizado en C++ en la Tarea 1. La clase **Rutina** provee métodos como `agregarEjercicio()`, `eliminarEjercicio()`, `calcularTiempoTotal()`, `contarPorTipo()`, `contarPorIntensidad()` y `getNivelString()`, los cuales son utilizados tanto para la generación de la rutina como para el resumen final de la sesión.

La clase **GestorRutinas** actúa como el **núcleo central del backend** y es el único punto de contacto entre la lógica del sistema y el frontend. Concentra el `ArrayList` de **ejercicios cargados desde el archivo CSV**, el objeto **Rutina** de la sesión actual y el **nombre del cliente**. El método `cargarEjercicios()` implementa la **lectura del archivo CSV mediante `BufferedReader`**, procesando línea por línea con `split(",")` para separar los siete campos de cada registro, y retorna un valor de tipo entero que permite al frontend distinguir entre cuatro situaciones: el valor 0 indica carga exitosa, el valor 1 indica que el archivo no fue encontrado lanzando `FileNotFoundException`, el valor 2 indica un error de formato numérico capturado mediante `NumberFormatException`, y el valor 3 indica cualquier otro error inesperado. El método `generarRutina()` recibe la cantidad de ejercicios de fuerza y cardiovasculares deseados junto con el nivel de intensidad, construye dos listas de candidatos filtrando del catálogo principal los ejercicios que cumplen simultáneamente con el nivel de intensidad solicitado y con la restricción de no repetición verificando que `ultimaSemana` sea distinto de 1, y selecciona aleatoriamente mediante la clase `Random` de Java removiendo cada ejercicio del pool para evitar repeticiones dentro de la misma sesión, retornando un booleano que indica si la generación fue exitosa. El método `descargarRutina()` recibe la rutina actual y la ventana padre, abre un `JFileChooser` para que el usuario elija la ubicación de guardado, y **escribe el contenido formateado línea por línea mediante `BufferedWriter`** incluyendo la información completa de cada ejercicio. Para diseñar este método se tuvo en cuenta la **separación de responsabilidades** entre la clase **GestorRutinas**, **Rutina**, y **Ejercicio**, donde cada uno se hace responsable de formatear su propia información, de modo que **Rutina** no necesita conocer la arquitectura interna de **Ejercicio**, ni **GestorRutinas** la de **Rutina**.

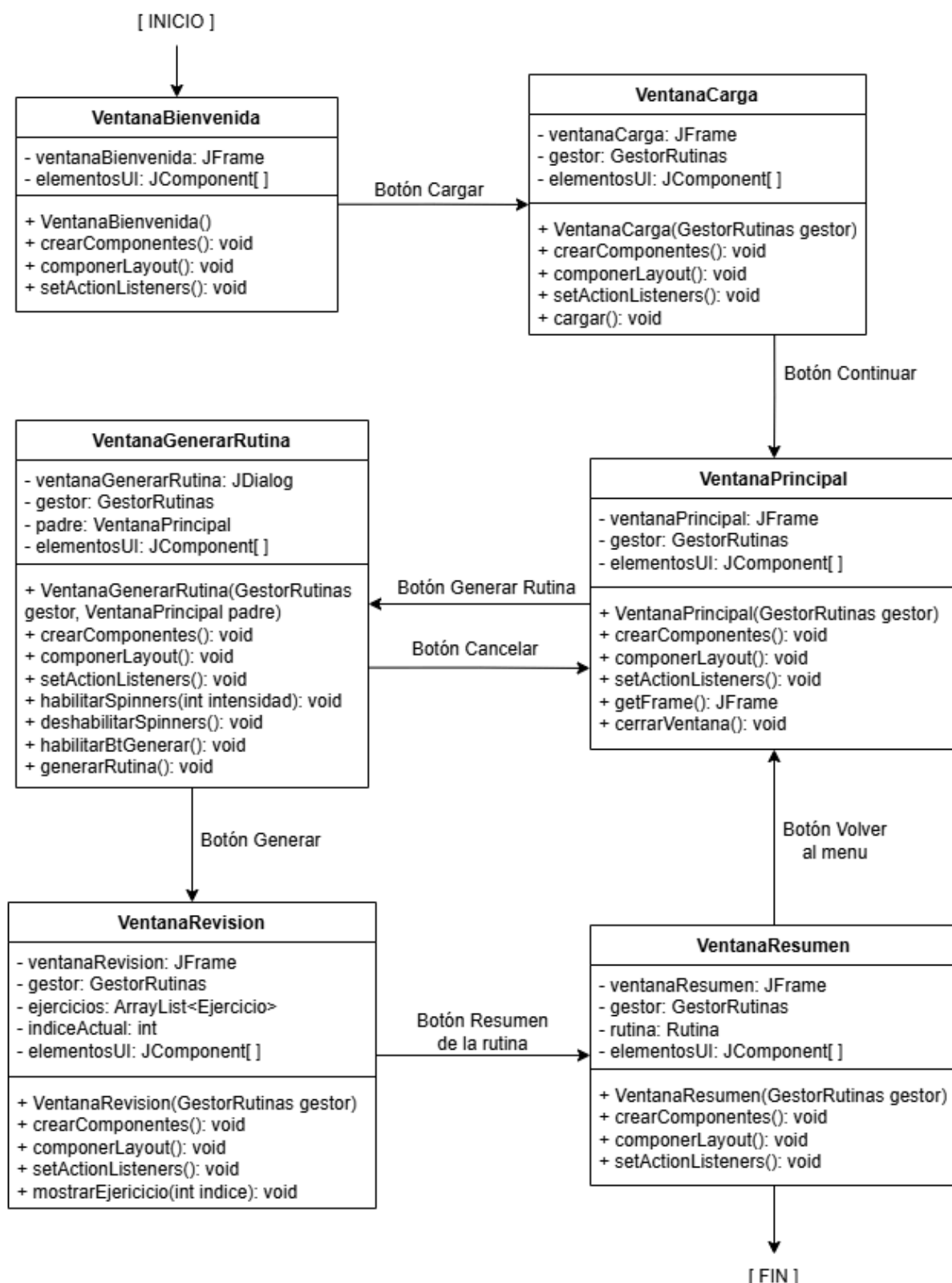
A continuación se muestra el **diagrama de clases** que representa de manera visual las clases del paquete backend descritas hasta el momento, junto con una representación menor de las clases del paquete frontend, que posteriormente se introducirán de manera más detallada.



Para la interfaz gráfica se utilizó la biblioteca **Swing** de Java, diseñando seis ventanas con una identidad visual coherente basada en fondos de gimnasio y una paleta de azul sobre fondos oscuros. La clase **Estilos** centraliza todas las constantes visuales del sistema en atributos estáticos finales, incluyendo la fuente Arial, una paleta de seis colores consistentes y cuatro recursos de imagen cargados mediante el método privado `cargarImagen()` que valida la existencia de cada recurso y detiene el sistema con un mensaje de error si alguno no se encuentra. Cada ventana del sistema sigue un patrón de construcción consistente organizado en tres métodos: `crearComponentes()` que instancia y configura visualmente cada elemento, `componerLayout()` que los posiciona mediante `GridBagLayout` con un objeto `GridBagConstraints`, y `setActionListeners()` que asigna el comportamiento a cada elemento interactivo en la interfaz.

VentanaBienvenida constituye el punto de entrada donde el usuario ingresa su nombre, validando mediante `isBlank()` y `length()` que el campo no esté vacío ni exceda los 30 caracteres antes de crear el `GestorRutinas` y abrir **VentanaCarga**. **VentanaCarga** ejecuta la lectura del CSV invocando `cargarEjercicios()` del gestor e interpreta el código de retorno mediante un switch para actualizar los labels de estado con mensajes y colores diferenciados, habilitando el botón Continuar únicamente cuando la carga retorna el valor 0. **VentanaPrincipal** despliega el saludo personalizado con el nombre del cliente, el total de ejercicios disponibles, el tiempo total del catálogo y seis tarjetas de estadísticas por tipo e intensidad construidas con HTML embebido en `JLabel` mediante las etiquetas `center` y `br`, además del botón para acceder a la generación de rutina. **VentanaGenerarRutina** se implementa como un `JDialog` modal que recibe como padre el `JFrame` interno de **VentanaPrincipal** mediante el método `getFrame()`, presenta un `JComboBox` con los cuatro niveles de intensidad incluyendo una opción inicial "Seleccionar...", dos `JSpinner` para las cantidades de ejercicios por tipo con límites dinámicos calculados mediante `cantTipoPorIntenRestringido()` del gestor, y habilita el botón Generar únicamente cuando la suma de ambos spinners es mayor a cero. **VentanaRevision** implementa la navegación ejercicio por ejercicio mediante el atributo `indiceActual` que se incrementa o decrementa en el método `mostrarEjercicio()`, que actualiza todos los labels y el `JTextArea` de descripción con `setLineWrap(true)` y `setWrapStyleWord` activo para respetar las palabras completas al saltar de línea, deshabilita el botón Anterior cuando `indiceActual` es cero cambiando su color a gris, y transforma el botón Siguiente en "Resumen de la rutina" con color claro cuando el índice alcanza el último ejercicio. **VentanaResumen** muestra el consolidado completo de la sesión con el nombre del cliente, el total de ejercicios, el tiempo total y seis tarjetas de distribución por tipo e intensidad, y ofrece tres botones: descargar la rutina como archivo de texto invocando `descargarRutina()` del gestor, volver al menú principal creando una nueva instancia de **VentanaPrincipal**, o salir del sistema mediante `System.exit(0)`.

A continuación se presenta una **extensión del diagrama de clases** que detalla el contenido del paquete frontend, incluyendo el flujo, de forma intuitiva, de las ventanas del programa a medida que el usuario interactúa con él.



Conclusión

El desarrollo de **SmartTrain** representó un salto cualitativo significativo respecto a la Tarea 1, evolucionando desde un sistema de consola en C++ hacia una **aplicación completa con interfaz gráfica, arquitectura en capas y comunicación controlada entre componentes**. El resultado es un sistema funcional, visualmente coherente y arquitecturalmente sólido que cumple con la totalidad de los requerimientos establecidos en el enunciado.

La decisión más relevante del desarrollo fue la **separación estricta entre los paquetes backend y frontend**, que permitió a ambos integrantes trabajar en paralelo sin conflictos, ya que cualquier cambio en la interfaz gráfica no afectaba la lógica central del sistema. Esta experiencia consolidó de manera práctica la comprensión de por qué el bajo acoplamiento es un objetivo fundamental en el diseño de software profesional.

La transición desde C++ hacia Java **implicó asimilar diferencias fundamentales más allá de la sintaxis**. La gestión automática de memoria mediante el Garbage Collector eliminó la necesidad de punteros, operadores delete explícitos, y destructores virtuales que fueron centrales en la Tarea 1, reemplazando el vector de punteros por ArrayList con un manejo mucho más seguro y limpio. El diseño de la interfaz gráfica con Swing **introdujo conceptos propios del paradigma orientado a eventos**, donde cada botón delega su comportamiento a un ActionListener y el sistema reacciona a las acciones del usuario sin necesidad de un bucle de control explícito como el do-while de la consola. El uso consistente del patrón crearComponentes(), componerLayout() y setActionListeners() en cada ventana demostró ser una decisión acertada que garantizó **orden y legibilidad en el código del frontend**. La centralización de estilos en la clase Estilos permitió **mantener coherencia visual** en las seis ventanas con mínima duplicación de código, y el uso de HTML embebido en JLabel simplificó considerablemente la presentación de datos con formato enriquecido sin necesidad de componentes adicionales.

En síntesis, **SmartTrain** no solo cumple con los objetivos funcionales del enunciado, sino que lo hace sobre una **base arquitectónica preparada para crecer**. Los aprendizajes obtenidos, desde la separación en capas y el diseño orientado a objetos hasta la construcción de interfaces gráficas intuitivas y el manejo de eventos, constituyen **conocimientos fundamentales** que trascienden el contexto académico y se proyectan directamente hacia el ejercicio de la ingeniería informática.